

Module 14: Introduction to ANN

Module Overview

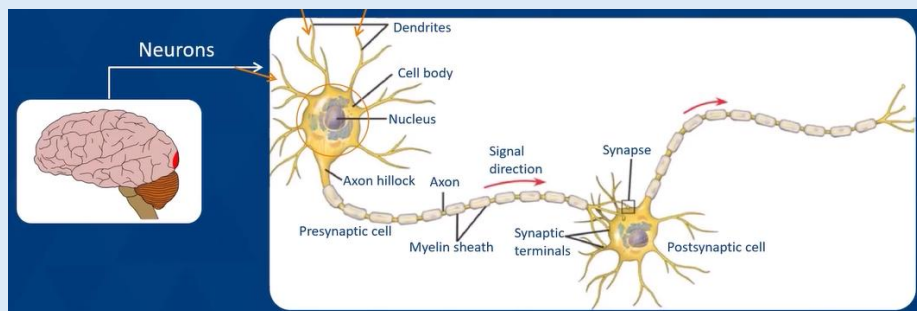
- Artificial neural networks replicate biological brain functions, enabling machines to learn and adapt through data-driven cognitive processes.
- Network architecture ranges from single-layer models to deep multi-layer systems, with neurons and activation functions guiding decision-making.
- Training involves adjusting weights and biases to optimise performance, forming the foundation for intelligent behaviour in AI systems.
- Keras library in Python supports practical implementation of neural networks, offering tools for building and experimenting with deep learning models.

- Compare biological and artificial neural networks
- Explore how ANNs mimic cognitive processes
- Understand single-layer to multi-layer structures
- Identify their role as ANN building blocks
- Learn key concepts and build a neural network using Keras in Python

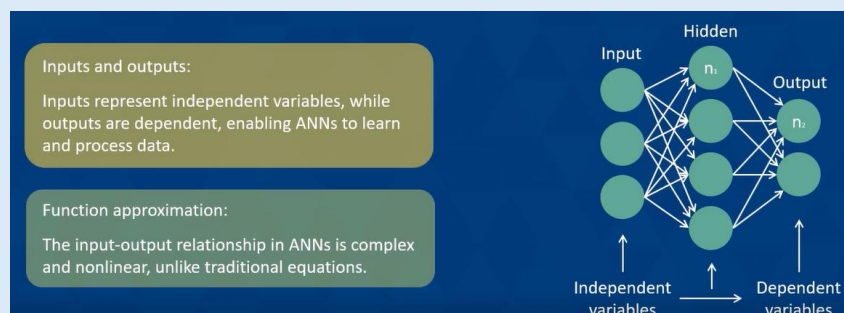
- Deep learning applications extend AI capabilities by enabling systems to recognise patterns, make predictions and solve complex problems autonomously.

What is a Neural Network (NN)?

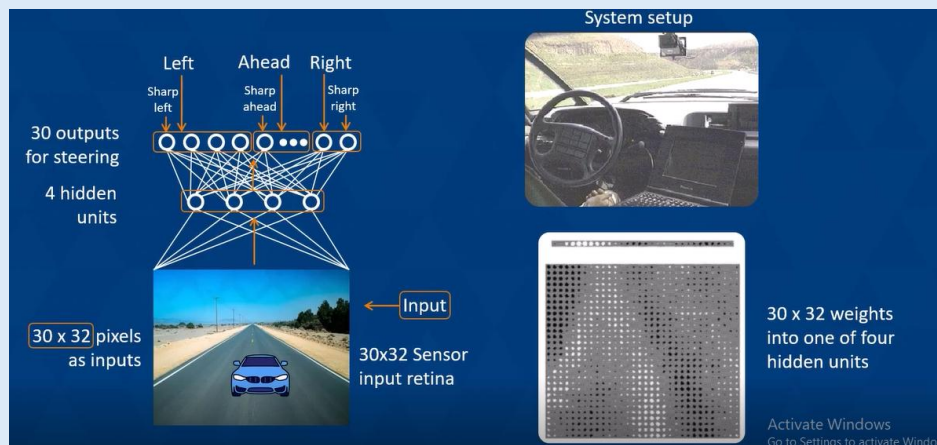
- Biological neurons serve as the foundational inspiration for artificial neural networks, enabling the simulation of cognitive processes.



- Artificial neural networks approximate complex non-linear functions by mapping inputs to outputs through layered processing units.
- A typical neural network architecture includes an input layer, one or more hidden layers, and an output layer for decision output.

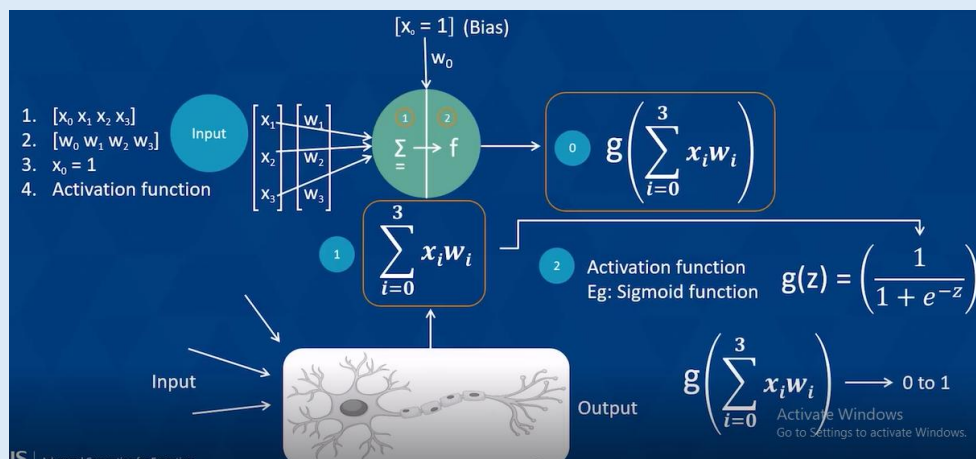


- Even small-scale neural networks with limited neurons can handle tasks like pattern recognition and real-time decision making.
- ALVINN, an early self-driving car system from 1989, demonstrated how simple neural networks could process visual input for steering control.



What is a Neuron in Artificial Neural Network?

- A neuron in an artificial neural network acts as the fundamental processing unit that receives inputs and produces outputs.
- Each neuron computes a weighted sum of its inputs, adds a bias term, and applies an activation function to generate an output.
- Activation functions introduce non-linearity into neural networks, enabling them to model and learn complex, real-world patterns.
- Weights, biases, and activation functions collectively determine the output of a neuron and are optimised during the training process.



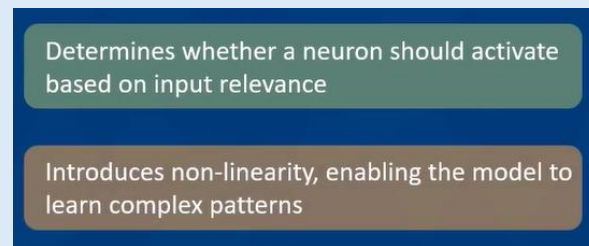
- The same computational principles apply across all neurons in different layers, allowing scalable architectures for advanced learning tasks.

All neurons in an ANN follow the same rules, enabling efficient scaling

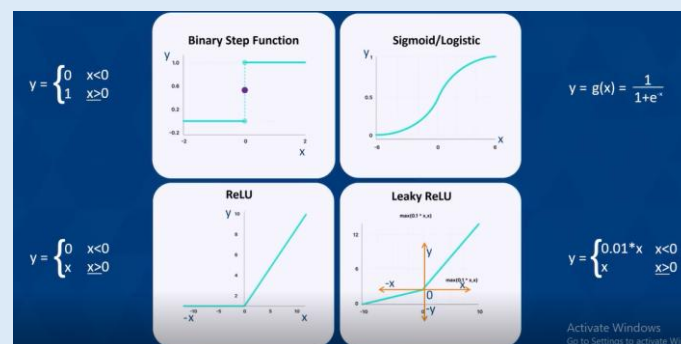
Stacking multiple layers enhances learning capability

What are the Different Activation Functions in ANN?

- Activation functions enable artificial neural networks to model complex, non-linear relationships in data such as images and speech.

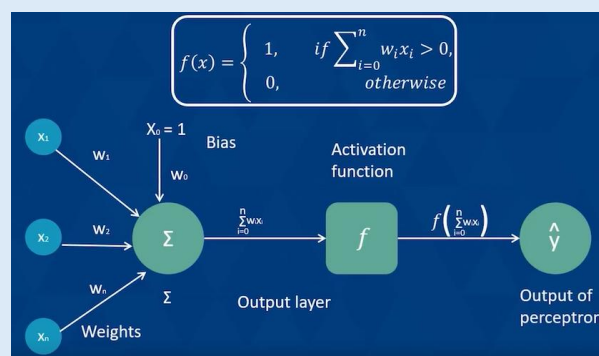


- Non-linearity introduced by activation functions allows neural networks to outperform linear models when handling real-world problems.
- Common activation functions include binary step, sigmoid, ReLU, and leaky ReLU, each transforming input signals differently.
- Sigmoid maps input to values between zero and one, making it suitable for binary classification and probability-based outputs.
- ReLU and its variant leaky ReLU help overcome issues like vanishing gradients and dead neurons during training of deep networks.

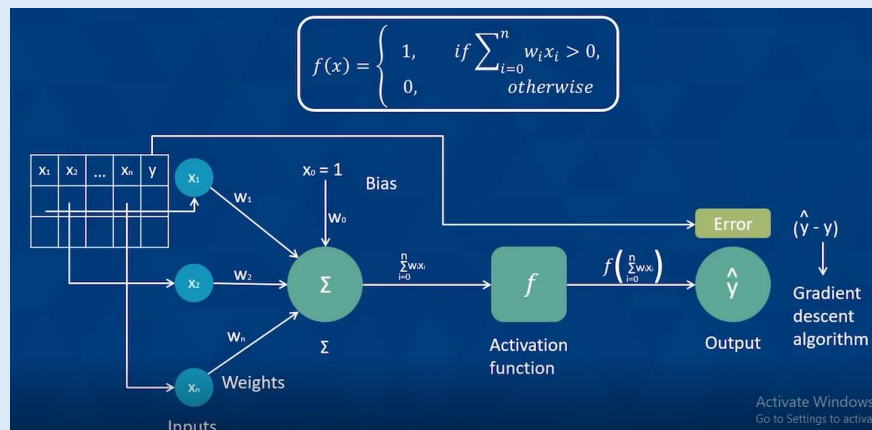


What is a Single-Layer Neural Network (Perceptron)?

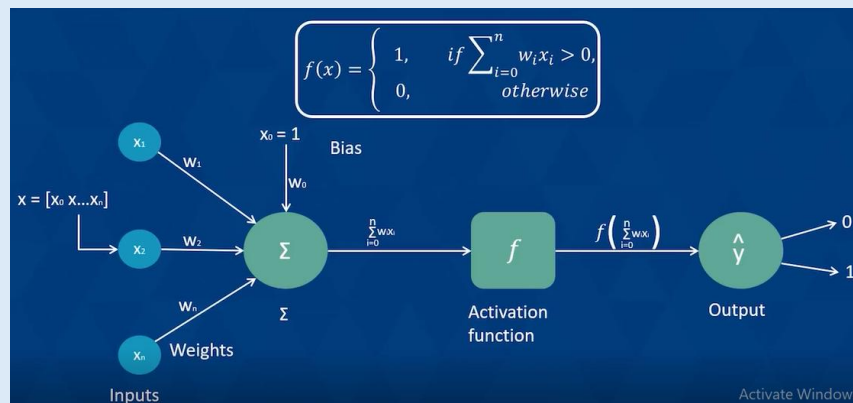
- A perceptron is the simplest type of neural network consisting of only input and output layers without hidden layers.
- Inputs are processed by multiplying with weights, adding a bias, and passing through an activation function to produce output.
- The step function is commonly used as the activation function, producing binary outputs based on a threshold value.



- Training a perceptron involves updating weights using gradient descent to minimise prediction error on labelled training data.

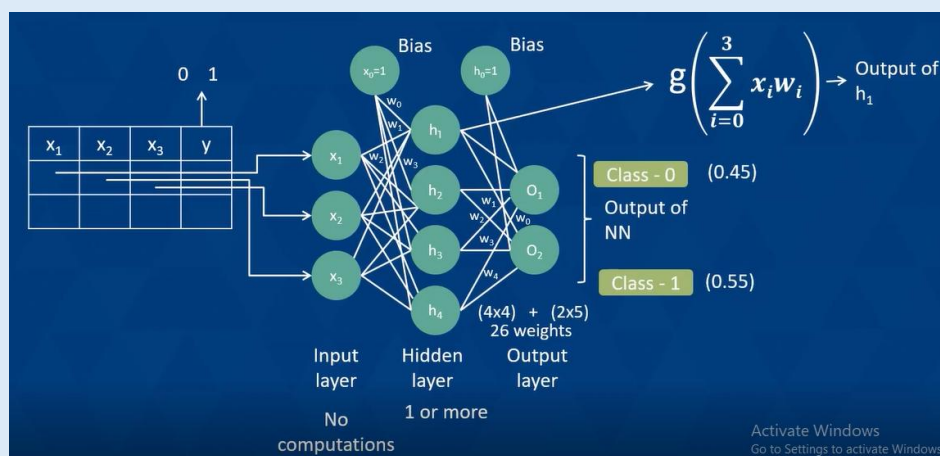


- Despite its simplicity, the perceptron provides a foundational understanding of how more complex neural network models function.

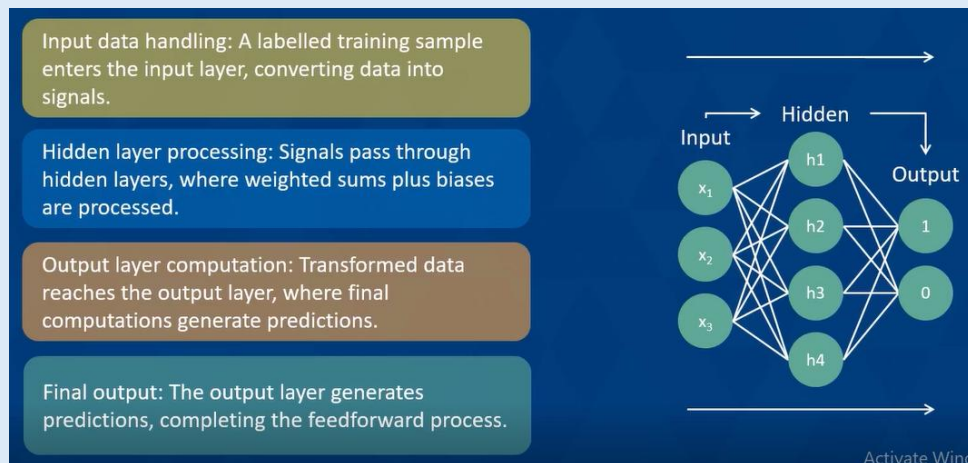


What is a Multi-Layer Neural Network?

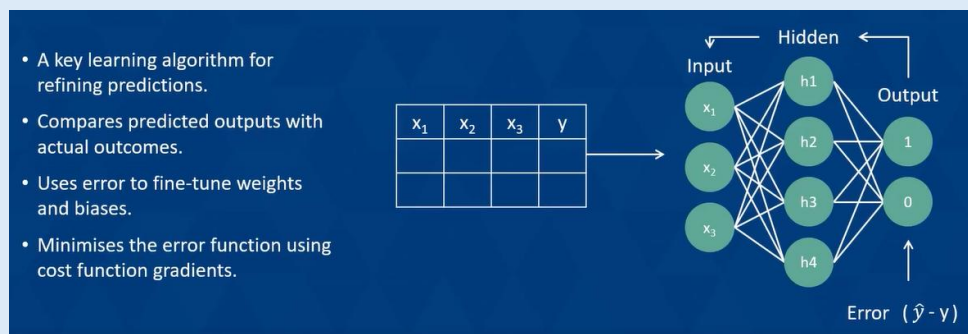
- A multi-layer neural network consists of input, one or more hidden layers, and an output layer to model complex tasks.
- Each layer transforms inputs using weights, biases, and activation functions, enabling non-linear learning and deep pattern recognition.



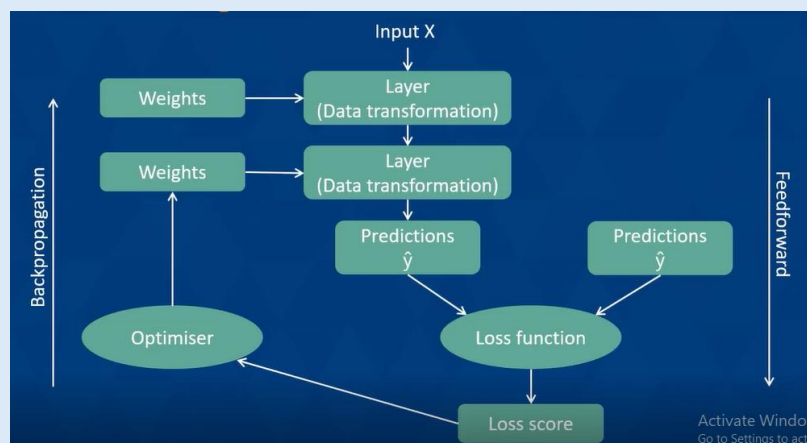
- Feedforward process ensures data flows linearly from input to output layer without cycles, enabling predictions for classification or regression.



- Backpropagation adjusts network weights by propagating the error backward from output to input, using gradient descent to reduce loss.



- The combination of feedforward computation and backpropagation learning allows the network to improve performance across multiple iterations.



Implement a Simple NN in Python Using Keras Library

- A neural network model is built using the Keras library to perform a regression task predicting California housing prices.
- The dataset includes features such as median income and house age, which are pre-processed and scaled for model efficiency.

```
{
  "cells": [
    {
      "cell_type": "code",
      "execution_count": 7,
      "metadata": {
        "id": "IFvd_rYk6ev"
      },
      "outputs": [],
      "source": [
        "# Import necessary libraries\n",
        "import numpy as np # Import Numpy for numerical operations\n",
        "import pandas as pd # Import Pandas for data manipulation\n",
        "from sklearn.datasets import fetch_california_housing # Import function to fetch California housing dataset\n",
        "from sklearn.model_selection import train_test_split # Import function to split data into training and testing sets\n",
        "from sklearn.preprocessing import StandardScaler # Import function for standardizing data\n",
        "from keras.models import Sequential # Import Sequential model from Keras for building neural network\n",
        "from keras.layers import Dense # Import Dense layer for fully connected neural network layers\n",
        "from keras.utils import plot_model # Import function to plot model architecture (optional)\n"
      ]
    },
    {
      "cell_type": "code",
      "execution_count": 8,
      "metadata": {
        "id": "1268061-w0c-t0c-w363-04671806a2"
      },
      "outputs": [],
      "source": [
        "# Load data from the California housing dataset\n",
        "housing = fetch_california_housing() # Fetch the dataset from sklearn.datasets\n",
        "X = pd.DataFrame(housing.data, columns=housing.feature_names) # Create a DataFrame for features with column names\n",
        "y = housing.target # Assign the target variable (median house value) to y\n",
        "print(X.shape)\n",
        "print(X.head())\n",
        "print(y.shape)\n",
        "print(y[5])\n"
      ]
    }
  ]
}
```

- A sequential neural network architecture is created with dense layers of 256, 128, and 64 neurons, using ReLU activation functions.

```
{
  "cells": [
    {
      "cell_type": "code",
      "execution_count": 16,
      "metadata": {
        "id": "1268061-w0c-t0c-w363-04671806a2"
      },
      "outputs": [
        {
          "output_type": "text",
          "text": [
            "Layer (type)                 Output Shape              Param\n",
            "-----\n",
            "dense_1 (Dense)              (None, 256)               204\n",
            "-----\n",
            "dense_2 (Dense)              (None, 128)              3280\n",
            "-----\n",
            "dense_3 (Dense)              (None, 64)               820\n",
            "-----\n",
            "dense_4 (Dense)              (None, 1)                65\n",
            "-----\n",
            "Total params: 4081 (17.00 KB)\n",
            "Trainable params: 4081 (17.00 KB)\n",
            "Non-trainable params: 0 (0.00 KB)\n"
          ]
        }
      ],
      "source": [
        "# Create a sequential model\n",
        "model = Sequential()\n",
        "model.add(Dense(256, activation='relu'))\n",
        "model.add(Dense(128, activation='relu'))\n",
        "model.add(Dense(64, activation='relu'))\n",
        "model.add(Dense(1, activation='linear'))\n",
        "model.compile(optimizer='adam', loss='mse')\n",
        "model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.2)\n",
        "model.evaluate(X_test, y_test)\n",
        "model.save('model.h5')\n"
      ]
    }
  ]
}
```

- The model is compiled using the Adam optimiser and mean squared error as the loss function, suitable for continuous value prediction.

```
{
  "cells": [
    {
      "cell_type": "code",
      "execution_count": 17,
      "metadata": {
        "id": "1268061-w0c-t0c-w363-04671806a2"
      },
      "outputs": [
        {
          "output_type": "text",
          "text": [
            "loss: 0.3273\n",
            "Mean Squared Error on Test Data: 0.32724206697208\n"
          ]
        }
      ],
      "source": [
        "# Evaluate model on test data\n",
        "loss_test = model.evaluate(X_test, y_test)\n",
        "print('Mean Squared Error on Test Data: (mse_test)')\n"
      ]
    },
    {
      "cell_type": "code",
      "execution_count": 18,
      "metadata": {
        "id": "1268061-w0c-t0c-w363-04671806a2"
      },
      "outputs": [],
      "source": [
        "# Compile the model with Adam optimizer and mean squared error loss function\n",
        "model.compile(optimizer='adam', loss='mse')\n",
        "# Train the model on training data\n",
        "# - epochs: Number of times the entire dataset is passed forward and backward through the neural network\n",
        "# - batch_size: Number of samples per gradient update\n",
        "# - validation_split: Fraction of training data to be used as validation data\n",
        "model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.2)\n"
      ]
    }
  ]
}
```

- Training involves 10 epochs with validation on a subset of data, allowing the model to adjust weights and minimise prediction error.

THE END